



EDE PLATFORM · ARCHITECTURE APPROACH

NVOCC / Shipping Agency Domain Enablement

How to enable NVOCC and liner-agency operations on the existing platform — what to reuse, what to extend, what to build, and the two things that must never be done.

Verdict: Build NVOCC as **new modules inside the existing logistics domain** — not as a peer domain, and not behind a new `logistics_base` domain. Roughly **80% of the 33 SOW deliverables are configuration, seed data, and extension slices** over assets that already ship.

Two gates before Phase 1: one approval-gated platform change (a second scope dimension for Principal), and four SOW deliverables that belong to platform modules that have not shipped yet.

01 The packaging decision

The question posed was: should we split a shared **logistics_base** enabler domain, so a freight-forwarding customer activates **logistics**, an NVOCC customer activates **nvocc**, and both depend on the base? The answer is no — and the reason is structural, not stylistic.

~80%

SOW IS CONFIG / EXTENSION

2

NEW MODULES NEEDED

1

PLATFORM CHANGE (GATED)

0

NEW DOMAINS REQUIRED

The outcome you want is already available

Activation granularity in this platform is **per module**, not per domain. A freight-forwarding tenant and an NVOCC tenant simply receive different **ACTIVE_MODULES** lists inside the same logistics domain. No split is needed to package the two products separately.

Why a separate **logistics_base** domain does not solve it

Every app key is derived from folder truth as `<domain_type>.<app_name>`. A third domain package therefore produces keys like **logistics_base.masters**, and an NVOCC module would declare a dependency on it. That is **still a domain → domain dependency** — precisely the thing the architecture forbids: *domains never depend on other domains; a concept two domains share belongs in the platform*. Renaming the package changes the string, not the topology. The rule bites identically.

⚠ The framework will not stop you

Dependency validation only checks that a declared dependency is **already loaded**, and domain load order is settings-driven. So a three-domain arrangement **boots successfully**. The constraint is architectural governance and code review — not a runtime guard. Shipping it anyway silently ends the one-directional dependency graph, which is the exact property that keeps modules independently activatable.

The three candidate shapes, scored

#	Shape	Consequence	Verdict
A	NVOCC as new modules inside the logistics domain	Purely additive. Shared masters stay where they are. Dependency graph stays one-directional. Per-tenant packaging via the module list.	RECOMMENDED
B	NVOCC as a peer domain, shared masters lifted to the platform	Legal, but requires relocating ~42 models out of the logistics masters module, rewriting every foreign key across 11 modules, migrating the tables, and re-governing the model-placement policy that currently assigns trade locations, equipment types, and incoterms to the logistics domain.	ONLY IF FORCED
C	A third logistics_base domain that both depend on	Does not fix anything. Produces exactly the forbidden domain → domain edge under a new name, while adding a package, a settings file, and an ordering constraint.	REJECT

The base module already exists — it is just not called that

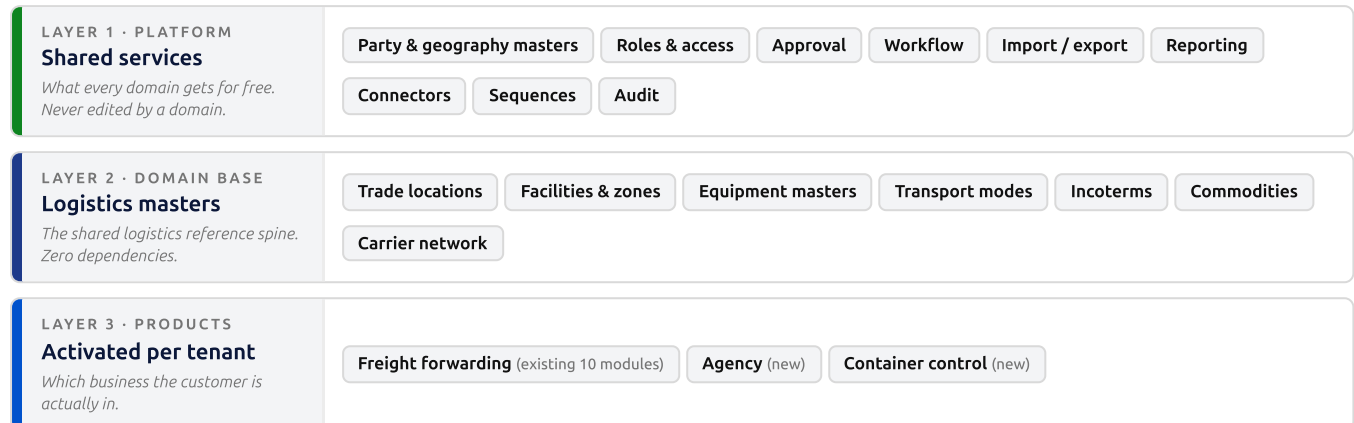
The logistics masters module already declares **no dependencies at all**, and every other logistics module depends on it. It is the domain's base layer in everything but name. Nothing needs to be created, renamed, or moved to obtain the "shared enabler base" the SOW is reaching for.

One correction the SOW needs

SOW §6 assumption 2 states NVOCC is "a new business domain that depends only on shared platform capabilities and never on a sibling business domain." Read strictly, that assumption **contradicts the SOW's own reuse intent** — a peer domain may not reuse the logistics masters. Restate it as: "NVOCC is delivered as new modules within the logistics business domain, reusing that domain's base masters and the shared platform layer." Same reuse guarantee, same clean graph, no lift required.

02 The shape to build

Two new modules, both inside the logistics domain, each owning a coherent slice of the SOW. Everything they need from below already exists.



All three layers sit in one dependency direction — upward only. A product module may read from the base and the platform; neither ever reads back.

The two new modules

Module	Owns	SOW deliverables
Agency <code>logistics.agency</code>	Principal master and its status lifecycle; principal context switching and data scoping; branch / entity / office entitlement; agency admin configuration switches; numbering rules; the integration-partner registry; governance reports; the master-data readiness gate.	D1.1–D1.8, D4.1–D4.5, D5.4, D5.5
Container control <code>logistics.container_control</code>	Container attributes on the existing equipment registry; ownership-type master; movement-event external-code crosswalk; agency stock-location seeds; check-digit and duplicate validation.	D2.1–D2.8

Where the party / charge / finance deliverables actually land

Deliverable	Correct owner	Why not an NVOCC module
Customer & vendor extensions, agent master	<code>logistics.agency</code>	Agency-specific policy — genuinely ours.
Charge-code master	Logistics masters (the domain base)	Pricing, booking, shipments and documentation all need it. Building it inside an NVOCC module would make four existing modules depend on NVOCC.
Tax / TDS setup	Platform localization	Tax is finance-wide, not logistics-specific. Two or more domains need it, so it is a platform concern by the placement test.
Finance-mapping references	Platform (layer decision open)	Every domain hands off to external accounting. Burying the mapping in NVOCC guarantees a second copy later.
Regional control framework, India & UAE packs	Platform localization + country packs	The country-scope mechanism is a cross-cutting SDK. NVOCC is a consumer of it, never its owner.

Keep the integration registry inside Agency

The integration-partner registry is governance configuration with no independent lifecycle, so a third module for it would be pure overhead. Promote it to its own module only when live message execution — a later BRD — gives it a state machine, retries, and its own audit trail.

03 What already exists

The SOW's design intent — *"reuse the existing freight foundation"* — is achievable to a greater degree than the SOW itself assumes. This is the audited inventory.

Capability the SOW needs	What is already shipped	State
PLATFORM LAYER — SHARED BY EVERY DOMAIN		
Party master (customer / vendor / agent / carrier)	Universal party master with role assignments, multiple addresses, optional company ownership, and tenant-customisable extra fields.	HAVE
Legal entity, branch, office	Organization master with a parent link — branch is already modelled . The access enricher already resolves an actor's organizations, branches, and permitted organization list.	HAVE
Role- and action-wise access control	Roles, permissions, role bindings, access grants, team roles, and a decision log. Row-level record rules with actor-attribute substitution.	HAVE
Audit trail with old / new values	Field-change log, access-binding change log, approval event log, configuration change log, plus a per-record activity timeline.	HAVE
Maker-checker approval	Full engine: rules, policy sets, flow templates, step definitions, cases, tasks, immutable decisions, delegation and escalation policies — driven by declarative guard expressions.	HAVE
Status lifecycles	Workflow engine with definitions, stages, transitions, guards and transition actions. Status-bearing fields are engine-guarded at the data layer.	HAVE
Bulk import with staging & row errors	Templates, per-column mapping, validators, import runs, and per-row error records — the staging-then-post model the SOW specifies.	HAVE
Reports & KPIs	Declarative dataset blueprints compiled to SQL, a metric engine with dependency resolution, and a dashboard module.	HAVE
Numbering / sequences	Sequence service with prefix, year / month segmentation, padding, step and annual reset.	HAVE
External system identity	Connector registry with per-connector parameters — endpoint, auth, environment.	HAVE
Cross-module field & view additions	Model-extension SDK plus view inheritance: a consuming module adds fields and patches forms owned by another module, shipping its own migration.	HAVE
Scope-based data isolation	Declarative company scope: the data layer auto-injects an ownership filter on every read and fails closed when no scope context is present. Create-time stamping is automatic.	HAVE
Country-scoped extensions	The scope-predicate slot exists on every extension field but nothing reads it yet — the evaluator is planned, not shipped.	NOT YET
LOGISTICS DOMAIN BASE — SHARED BY EVERY LOGISTICS PRODUCT		
Container / equipment master	Physical equipment registry with identifier, type, category, transport mode, owner, operator, custodian, current facility, current status, current condition, last-movement timestamp, inspection date, damage flag, lease type, archive flag — roughly 90% of the SOW's container master .	HAVE
Equipment type / category / identifier masters	Seven seeded masters covering type, category, subcategory, identifier type, status, condition and usage status.	HAVE
Movement events	Movement-event type master plus recorded movement events against equipment usage.	HAVE
Yard / CFS / depot / terminal locations	Facility master with facility types, zones, zone types, and cargo-handling-location types.	HAVE
Ports and trade locations	Trade-location master, populated on demand by the reference-sync service rather than seeded.	HAVE
Carrier / provider / agent network	Carrier, carrier agent, provider and vendor-service masters.	HAVE
Segmented numbering rules	A shipment numbering-rule model that already segments by organization, branch, mode, type and year, with separator, padding, period reset and a counter sequence.	REUSE VIA LIFT
Charge-code master	Only transactional charge <i>lines</i> exist (rate slabs, booking / shipment / document charges). There is no charge-code master anywhere.	GAP

04 Deliverable map — Phases 1 & 2

Every SOW deliverable, mapped to the asset it reuses and the mechanism that closes the gap. **REUSE** means configuration or seed data only. **EXTEND** means an extension slice. **BUILD** means new engineering. **GATED** means blocked on a decision or another module.

D	Deliverable	Reuse & mechanism	Class
PHASE 1 — PRINCIPAL FOUNDATION & ACCESS CONTROL			
D1.1	Principal master	New operating-identity record carrying a link to the universal party master for legal entity, tax registration, addresses and contacts — mirroring how the organization master already shadows a party. Country, document prefix and external code all reference existing services.	BUILD
D1.2	Principal status lifecycle	Workflow definition, stages, transitions and eligibility guards — declared as data. Zero engine code. A shipped booking workflow is the working precedent.	REUSE
D1.3	Principal context switching	Mirrors the existing active-organization switch end to end: a runtime context slot, a switch action, a permitted-list on the user, and the frontend switcher. Needs a second scope dimension — see §06.	GATED
D1.4	Principal-wise data scoping	Same mechanism. Critically, the existing scope filter already runs inside the data-access layer and fails closed — which is exactly what the SOW's highest-severity risk demands.	GATED
D1.5	Branch / entity / office entitlement	Branch is already the organization master's parent-child relation; the access enricher already emits an actor's organizations, branches and permitted list. Principal × branch grants are rows.	REUSE
D1.6	Access-control model	Roles, permissions, bindings, grants and the decision log — all shipped. The deliverable is a roles file and a permission table for the SOW's ten roles.	REUSE
D1.7	Audit trail	Field-change, access-change, approval-event and configuration logs plus the record timeline. Verify the import path records source and reason; no new store.	REUSE
D1.8	Duplicate prevention — principal	Field-level uniqueness, composite keys and pre-create validation hooks are framework-native.	REUSE
PHASE 2 — CONTAINER & EQUIPMENT CONTROL			
D2.1	Container type master	The equipment type, category, subcategory and identifier-type masters are shipped and seeded. Add the ISO type code as an extension field from the new module.	EXTEND
D2.2	Equipment ownership setup	Owner, operator, custodian and lease type already exist on the equipment record. Add an ownership-type master for the shipper-owned / carrier-owned / co-loader / third-party typology, plus one reference field.	BUILD
D2.3	Container master	The equipment registry is already ~90% of this deliverable. Missing only: principal link, check digit, lease reference, and reefer attributes. One extension class closes it.	EXTEND
D2.4	Container status lifecycle	Status and usage-status masters plus the workflow engine and its guard expressions. The SOW's status set and its two blocking rules are seed rows and guards.	REUSE
D2.5	Inventory movement mapping	Movement-event types and recorded events exist. The external-code crosswalk is new — and should be a separately versioned mapping record, because the SOW requires downstream consumers to read it read-only against a version.	BUILD
D2.6	Yard / CFS / depot master	Facility master, facility types, zones and cargo-handling types are all shipped. Depot, empty-park and repair-yard are seed rows .	REUSE
D2.7	Terminal / ICD / port extension	Trade-location and facility masters exist. Port, terminal, customs-station, message and port-community codes plus gate hours attach as an extension.	EXTEND
D2.8	Duplicate prevention + check digit	Uniqueness is native; the configuration switch and per-row import error reporting both exist. The check-digit algorithm is a small validator — register it with the import validator registry too , so bulk loads report bad rows instead of rejecting whole files.	BUILD

05 Deliverable map — Phases 3, 4 & 5

D	Deliverable	Reuse & mechanism	Class
PHASE 3 — PARTY, CHARGE & FINANCE-MAPPING EXTENSIONS			
D3.1	Customer extension	The universal party master already carries roles, addresses and tenant-customisable fields. Credit control, release hold, free days, billing party and KYC belong in side-car profile records referenced by one field each — not as a dozen flat columns on a master every domain shares.	EXTEND
D3.2	Vendor extension	The party role master plus the vendor-service model cover this entirely. The nine agency vendor roles are seed rows — no code at all .	REUSE
D3.3	Agent / partner master	Carrier, carrier-agent, provider and party masters exist. Principal link, service capability and commission rule attach as an extension.	EXTEND
D3.4	Charge master extension	No charge-code master exists anywhere — only transactional charge lines. Build the master in the logistics base , where pricing, booking, shipments and documentation can all use it, then extend it from Agency for agency billing flags.	BUILD (base)
D3.5	Tax / TDS setup	Tax is finance-wide, therefore platform-layer. Belongs to the localization module, which has not started .	NOT OURS
D3.6	Finance-mapping reference	A mapping master is legitimate — the standing decision is that charges live on transactional records and accounting leaves via connectors. But every domain hands off, so the mapping is platform-layer. Layer decision required .	DECISION
D3.7	Duplicate prevention — parties & charges	Same native machinery as D1.8.	REUSE
PHASE 4 — ADMIN, NUMBERING, INTEGRATION & GOVERNANCE			
D4.1	Numbering rules service	The sequence service plus an existing shipment numbering-rule model that already segments by organization, branch, mode, type and year with reset policy. Lift that resolver to a shared layer and migrate shipments onto it , then NVOCC configures rows — do not copy the model.	LIFT
D4.2	Admin configuration	Configuration store, typed accessors, organization-scoped values and a change log are shipped. Deliverable is nine switch keys plus one settings screen.	REUSE
D4.3	Integration partner registry	The connector registry already holds endpoint, auth and environment. New: the partner × message-type × direction matrix, referencing connectors — never re-modelling endpoints or credentials.	BUILD
D4.4	Maker-checker approval	The approval engine covers rules, policy sets, flow templates, steps, cases, tasks, decisions, delegation and escalation. Deliverable is flow data for six critical master classes .	REUSE
D4.5	Data import / export	Templates, column mapping, validators, runs and per-row error records all exist — the staging-then-post contract is already the platform's model. Deliverable is templates plus export permissions.	REUSE
PHASE 5 — REGIONAL PACKS, REPORTS & READINESS			
D5.1	Regional control framework	The per-field country-scope slot exists but no evaluator reads it . This is a platform SDK deliverable, currently not started — and a hard prerequisite for D5.2 and D5.3.	NOT OURS
D5.2	India control pack	Country-pack extensions gated by the country-scope predicate. Ships from a country-pack module, not from an NVOCC module .	NOT OURS
D5.3	UAE control pack	Identical pattern, second pack.	NOT OURS
D5.4	Reports & KPIs	Declarative dataset blueprints plus the metric engine and dashboards. The single largest "looks like code, is actually configuration" win in the SOW — fourteen reports become blueprint rows, not a reporting engine.	REUSE
D5.5	Master-data readiness gate	Compose from shipped engines: one metric per completeness dimension, a guard expression for the gate, and a workflow transition that will not fire until every dimension passes.	BUILD

14

CONFIG / SEED ONLY

5

EXTENSION SLICES

8

NEW ENGINEERING

4

NOT NVOCC WORK

2

PLATFORM-GATED

06 The one platform change

Principal context switching and principal-wise data scoping are the only architecturally hard deliverables in the SOW. Everything else is additive. This section is what needs approving, and why.

What the platform already does — for exactly one dimension

A model declares that it is company-owned, and from then on the data layer **automatically injects an ownership filter into every read**, stamps the owner on every create, and **fails closed** — no scope context means no rows, never all rows. The runtime context carries the active organization, and admin tooling opts out explicitly and traceably. This is a mature, correct implementation of exactly the guarantee the SOW's highest-severity risk asks for: *"enforce scoping at the data-access layer, not just the UI."*

Principal is a **second, orthogonal dimension**. A record belongs to an organization *and* is transacted under a principal. The two are independent, and both must filter.

OPTION 1 · GENERALISE

RECOMMENDED

Refactor the single company-scope concept into a **registry of named scope dimensions**, so organization, principal, and any future dimension share one implementation.

- One filter-injection path, one fail-closed rule, one create-time stamp, one documented bypass.
- Satisfies the SOW's top risk *by construction*, not by discipline.
- Every future dimension is then free.
- **Cost:** touches the runtime context and the data-layer filter — kernel files.

OPTION 2 · RULES ONLY

FALLBACK

Row-level record rules already substitute actor attributes into their filter expressions, and are merged into every read.

- But the actor-enrichment output keys are **fixed**; a domain module cannot contribute a new one.
- So this *still* needs a platform change — a smaller one: an extension point on the enricher.
- Leaves scoping enforced by data rows rather than by the kernel, and **does not stamp the principal on create**.
- Weaker guarantee against the same risk.

⚠ Approval gate — this is not a normal change

Modifying the runtime context or the data-layer filter is a **framework-shape change**. Governance requires explicit approval before any edit to the kernel, and this should be raised as a **platform roadmap change with its own tests** — never folded silently into the NVOCC build. Present the failure, the root cause, the two paths, and the proposed change; then wait.

⊖ Terminology hazard — the word "principal" is already taken

In this platform, the runtime **principal is the authenticated actor** — the logged-in user, used by every access check. In NVOCC, a **Principal is the carrier / operating identity** the agency transacts under. These are unrelated concepts colliding on one word, and confusing them would wire access control to a shipping line.

Binding naming rules: never add a context attribute named `principal` meaning the carrier. Use `active_principal_id` for the context slot, a `principal`-suffixed model key for the record, `principal_id` for the reference field, and `allowed_principal_ids` for the permitted list. State the collision in the module docstring and in the roadmap so no one wires the wrong one.

Recommendation

Take **Option 1**, raised and approved as a platform change **before Phase 1 starts**. If approval is withheld, Option 2 unblocks Phase 1 — but record explicitly that the SOW's top risk is then mitigated by configuration discipline rather than by the framework, and that cross-principal access tests become the only real defence.

07 Mechanisms & build sequence

Which mechanism for which job

Need	Mechanism
Add a field to a model owned by another module — even inside the same domain	The model-extension SDK, with the column migration shipping from the consuming module.
Show that field on the owner's form or list	View inheritance — patch the owner's view by reference from your own view file. Never edit theirs.
Gate a field to one country	The country-scope predicate. Evaluator not yet shipped — plan around it.
Let tenants add their own fields without code	Enable customisable properties on the model — a JSON column plus a validator, no schema change per tenant.
Soft-archive instead of delete	Declare an archive flag; the data layer hides archived rows and the UI renders the toggle automatically.
Isolate rows by organization	Declare the company scope on the model. Filtering and create-time stamping become automatic.
Restrict visibility by role	Row-level record rules with actor-attribute substitution — data, not code.
Status lifecycle with gated transitions	Workflow definition, stages, transitions and guard expressions — declared as data.
Maker-checker on a master change	Approval rule plus flow template, with a guard expression selecting which changes need review.
A behaviour switch an admin can flip	A configuration key read through the typed accessor — never a constant in code.
Bulk load with validation before posting	An import template with column mapping and validators; errors land per row.
A report or KPI	A dataset blueprint plus a metric — configuration, not a reporting module.
One module produces work another consumes	The producer owns the boundary record; each consumer ships its own reference field, its own command, its own button, and emits events back. No bridge module.

Same domain is not the same module

Adding a container attribute to the equipment registry means one module writing into another module's model. That goes through the extension SDK **even though both live in the logistics domain**. Editing the owner's source file is the violation regardless of whether the two modules share a domain.

Build sequence

Step	Action	Detail & gate
0	Close the blocking decisions	Packaging shape, the platform change, ownership of the four non-NVOCC deliverables, the charge-master home, and the finance-mapping layer. See §08.
1	Roadmap first	Author the per-module roadmap with every requirement, business rule and acceptance criterion mapped to a work stream before any code . The SOW's requirement IDs become the traceability spine.
2	Scaffold each module	Package init that imports models; a manifest declaring fully-qualified dependencies; folders for models, views, data, demo, migrations and tests.
3	Activate explicitly	Append the two modules to the domain's active-module list, after their dependencies . Nothing is auto-discovered; boot fails loudly on a wrong order.
4	Declare models	Dotted keys, correct relational suffixes, master records instead of hardcoded value lists, archive flags where archiving applies, scope declarations where ownership applies.
5	Ship extension slices	One extension per target model, each with its own migration and its own view patch, all from the consuming module.
6	Generate migrations	Always via the generator, per module. If output is wrong, fix the model and regenerate. Never hand-author a revision.
7	Ship the data layer	Roles and permissions, record rules, workflow definitions, approval flows, sequences and numbering rules, configuration switches, import templates, report blueprints, menus. This is where most of the SOW actually lands.
8	Ship demo data	A use-case document plus demo records, declared in the manifest and smoke-tested against a tenant. No feature is marked delivered without it.
9	Verify	Full suite green, plus cross-principal access tests in the acceptance suite — the SOW names these explicitly, and they are the only real proof that D1.4 holds.

08 What to do

Fourteen binding rules. Each one is the difference between an additive slice and a structural liability.

1 • Build NVOCC as modules inside the logistics domain

Two modules — agency and container control. The dependency graph stays one-directional and every module remains independently activatable. **Package products by the active-module list**, which is the lever that already exists for exactly this purpose.

2 • Treat the logistics masters module as the shared base, as-is

It already declares no dependencies and every logistics module already depends on it. **It is the "logistics_base" the SOW is reaching for.** Nothing needs creating, renaming or moving.

3 • Extend the equipment registry into the container master

It already carries identifier, type, owner, operator, custodian, current facility, status, condition, last-movement timestamp and lease type. Add only the principal link, check digit, lease reference and reefer attributes. **One extension class, one migration.**

4 • Use the extension SDK for every cross-module field and view addition

Declare the field from the consuming module, ship the column migration from the consuming module, and patch the owner's view by reference from your own view file. **This applies inside a single domain too** — module boundaries are what matter, not domain boundaries.

5 • Prefer a master record over a hardcoded value list

Ownership type, movement-event type, agency vendor role, release type — all of these are tenant-extensible reference data. A master is a row an admin can add; a hardcoded list is a release.

6 • Build the charge-code master in the logistics base, not in NVOCC

Pricing, booking, shipments and documentation all need it, and the platform's localization roadmap already names it as the canonical target for country-specific tax coding. **Building it inside an NVOCC module would make four existing modules depend on NVOCC.**

7 • Lift the numbering resolver before reusing it

A segmented numbering-rule model already exists for shipments, complete with organization / branch / type / year segmentation and reset policy. **Lift it to a shared layer, migrate shipments onto it, then configure NVOCC rows.** One implementation, two consumers.

Rules 8 to 14 continue overleaf

The first seven rules govern **where things live**. The remaining seven govern **how they are built and proven**.

08 What to do — build & proof rules

8 · Put agency policy in side-car profile records

Credit control, release hold, free days, billing party and KYC each become a small record referenced by one field — **not a dozen flat columns on the party master every domain shares**. Profiles are also reusable across customers; columns are not.

9 · Version the movement-event crosswalk

The SOW requires downstream systems to consume the crosswalk read-only, and names crosswalk drift as a live risk. Model it as versioned configuration owned here, so a mapping change is a new version rather than a silent edit under a running integration.

10 · Make every validation behaviour a configuration switch

Check-digit enforcement, credit-hold enforcement, principal-mandatory, maker-checker-required — the SOW specifies these as admin-configurable. Read them through the typed configuration accessor so tenants enable them independently, and **register the check-digit rule with the import validator registry** so bulk loads report bad rows instead of rejecting whole files.

11 · Deliver the reports as configuration

All fourteen governance reports plus the readiness KPIs are dataset blueprints and metrics. **Do not write a reporting layer**. This is the largest single saving in the SOW.

12 · Raise the scope-dimension change as a platform change, with approval

Generalise the one existing scope dimension into a registry so organization and principal share one filter path, one fail-closed rule and one create-time stamp. Get explicit approval first, give it its own roadmap entry and its own tests. **Never fold a kernel change into a domain build**.

13 · Name the carrier identity unambiguously

The runtime already uses "principal" for the authenticated actor. Use an explicitly qualified context slot, reference field and permitted-list name for the carrier identity, and document the collision where the next developer will read it.

14 · Write the cross-principal access tests as acceptance, not as afterthought

The SOW names data leakage between principals as its highest-severity risk and asks for these tests by name. They are the only evidence that scoping actually holds. **No principal-scoping deliverable is complete without them**.

09 What not to do

Each of these is a decision that looks reasonable in the moment and is expensive to unwind. The middle column is what it actually breaks. These first nine are **structural** — they change the dependency graph or the ownership of a model.

Do not	Why it breaks	Do instead
Create a separate NVOCC domain that depends on logistics	A domain → domain dependency. Boots fine, so nothing catches it — but the graph stops being one-directional and the modules stop being independently activatable.	Modules inside the logistics domain.
Create a third <code>logistics_base</code> domain	Produces exactly the same forbidden edge under a new name, while adding a package, a settings file and an ordering constraint. Solves nothing.	Use the existing masters module — it already is the base.
Create a parallel container model beside the equipment registry	Splits movement history across two tables, forks the status lifecycle, and breaks every existing equipment report and dashboard.	Extend the equipment registry.
Edit another module's model file to add your field	Creates an unowned dependency — the owner ends up referencing models it does not load. Reliably breaks test fixtures that register the owner without your module.	The extension SDK, with your migration.
Edit another module's view file to show your field	Same coupling, plus the field renders as broken whenever your module is not active.	View inheritance by reference from your own file.
Copy the shipment numbering-rule model for CRO / BL / manifest	Two near-identical resolvers that drift. The next document type makes it three.	Lift the resolver once, migrate the existing consumer, configure rows.
Flatten agency policy onto the shared party master	Pollutes a master every domain's forms render. Users in HR and finance see agency free-day fields they have no use for.	Side-car profile records, one reference each.
Build tax, TDS or the country framework inside an NVOCC module	These are finance-wide and cross-cutting. Building them in a domain module guarantees a second, incompatible copy the first time another domain needs them.	Platform localization; sequence it ahead of Phase 5 or descope.
Introduce a bridge module to glue NVOCC to freight forwarding	A fan-in hub that must depend on both sides. Couples their releases and makes ownership ambiguous. Requires explicit approval and is to be avoided by default.	Producer owns the contract; each consumer ships its own slice and signals back with events.

⚠ The first two are the ones to guard hardest

Both boot successfully and pass every test. Nothing in the framework will tell you they happened — only review will. Everything else on this list announces itself sooner.

09 What not to do — process & delivery

These nine are **process and delivery** failures. None of them changes the architecture, but each one costs a release, a customer escalation, or a phase-exit review.

Do not	Why it breaks	Do instead
Change the kernel to make principal scoping work, without approval	A framework-shape change made under domain-delivery pressure, with domain-shaped tests. Governance requires prior approval for exactly this reason.	Raise it as a platform change with its own roadmap entry and tests.
Overload the runtime "principal" to mean the carrier	Access control and carrier identity end up reading the same slot. The failure mode is silent and security-relevant.	Explicitly qualified names, documented at the module boundary.
Enforce principal scoping in the UI or in controllers	Every other read path — commands, reports, exports, background jobs, the API — leaks. The SOW names this as its top risk precisely because UI-level scoping looks like it works.	Enforce at the data-access layer, where the existing scope filter already runs and already fails closed.
Hardcode check-digit or credit-hold enforcement	The SOW specifies these as per-tenant switches. Hardcoding blocks legitimate bulk imports and forces a release to unblock a customer.	Configuration keys read through the typed accessor.
Reject a whole import file on a bad container number	Named as a live SOW risk. An operator loading two thousand containers cannot find the four bad rows.	Per-row errors through the import validator and staging report.
Hand-author a migration	Diverges from the model, breaks the next generated revision, and silently drifts across dialects. Not "just this once".	Fix the model, regenerate. If the generator fails, surface the root cause.
Write a reporting layer for the governance reports	Reimplements a shipped engine and forfeits its caching, permissions and dashboard integration.	Dataset blueprints plus metrics, as data.
Mark a deliverable done without demo data	The feature cannot be demonstrated at the phase-exit review the SOW commits to, and the next developer has no working example.	Ship the use-case document and demo records, and smoke-test them.
Seed trade locations from a file	The trade-location master is populated on demand by the reference-sync service. A seeded copy goes stale and then conflicts.	Let the sync service populate it.

10 Decisions to close, and risks to watch

Five decisions block Phase 1. Four of them are scope questions the SOW's own change-management clause exists to handle; one is an engineering approval.

Blocking decisions

#	Decision	What is being asked	Owner
1	Packaging shape	Approve modules-inside-logistics, and correct the SOW assumption that describes NVOCC as a peer domain — it contradicts the SOW's own reuse intent.	Eng Lead + PO
2	Scope-dimension platform change	Approve generalising the one existing scope dimension into a registry, as a platform change with its own roadmap entry — or accept the weaker rules-only fallback and record the residual risk.	Eng Lead
3	The four non-NVOCC deliverables	Tax / TDS and the three regional deliverables depend on a platform module that has not started. Either sequence it ahead of Phase 5, or move those four out of scope under the change-management clause.	PO
4	Charge-master home	Confirm the charge-code master is built in the logistics base as a shared master. It is a pre-existing gap that NVOCC surfaced, not an NVOCC deliverable.	Eng Lead
5	Finance-mapping layer	Decide where the mapping master lives, against the standing decision that this platform holds no accounting domain and hands off through connectors.	Eng Lead + PO

Risks specific to this enablement

Risk	Severity	Mitigation
A peer NVOCC domain ships anyway and takes a dependency on the logistics base	HIGH	Nothing catches it at runtime — dependency validation only checks load order. Close it in review, and add a boot-time assertion that no domain app depends on another domain's app key.
Principal scoping enforced above the data layer, leaking across principals	HIGH	Enforce in the data-access filter, which already fails closed. Cross-principal access tests are acceptance criteria, not optional coverage.
The two meanings of "principal" get wired together	MEDIUM	Binding naming rules in §06; documented at the module boundary; enforced in review.
Container control forks a parallel container model	MEDIUM	Extend the equipment registry. A parallel model splits movement history and breaks existing reports — recoverable only by migration.
Numbering rule copy-pasted instead of lifted	MEDIUM	Lift-then-migrate is a prerequisite of D4.1, not a follow-up. Sequence it that way in the roadmap.
Phase 5 starts before the platform localization module exists	MEDIUM	Three deliverables have a hard prerequisite that has not started. Resolve as decision 3 above before Phase 4 exit.
Agency policy fields flattened onto the shared party master	MEDIUM	Side-car profiles with one reference each; reviewed at model-design time, when it is still cheap.
The crosswalk drifts from the codes downstream integrations expect	MEDIUM	Versioned configuration owned here; downstream consumes a pinned version read-only.

The bottom line

NVOCC does not need a new domain, and it does not need a new base domain. It needs **two additive modules, one approved platform change, one shared master that was already missing**, and the discipline to deliver the remaining fourteen deliverables as configuration rather than code. Done that way, adding NVOCC changes nothing about how freight forwarding works — and adding the next product after it changes nothing about NVOCC.